SwiftLog Technical Report

Machine Learning Track | Mission ML05: Intelligent Logistics Prediction & Supply Chain Optimization

1. Purpose

This report documents the machine-learning workflow for four logistics tasks using the supplied SwiftLog educational datasets. The workflow includes data quality review, feature engineering, three candidate algorithms per task, held-out evaluation, limited hyperparameter tuning, permutation importance, saved model bundles, supplementary exploratory analysis, baseline comparisons, and a Streamlit prototype. This is an educational prototype, not a production-ready decision system.

2. Data preparation

Source tables were loaded from `Data/Original_Datasets`. Text fields were standardized, exact duplicate rows were removed from processed copies, and row counts, duplicate counts, and missing cells were recorded. Related order, item, product, customer, shipment, warehouse, and return tables were joined where required. Processed tables and engineered features are saved under `Data/Processed_Datasets`.

Supplementary EDA examined column-level missingness, numeric feature distributions, numeric correlations where multiple numeric columns were available, target distributions for classification, and IQR-based outlier summaries. Charts and CSV summaries are stored in Reports.

3. Targets and feature design

Delivery delay: positive when `DelayDays > 0`; DelayDays and post-event delivery status are excluded from predictors. **Product return:** positive when an OrderID appears in Returns; order timing, city, and order-item aggregates are used. **Demand forecasting:** weekly product quantity is the target, using lagged prior-week demand, a four-week lag, and a rolling mean of the preceding four weeks. **Customer value:** next-month revenue is the target, using customer activity from the preceding three months.

4. Evaluation design

Classification tasks use stratified random 80/20 train/test splits and report accuracy, precision, recall, F1, and ROC-AUC. Demand and customer-value tasks use chronological train/test splits. Regression reports MAE, RMSE, and R-squared. Tuning is limited and results are experimental. The supplementary baseline script constructs its own splits; identical test-row alignment has not been independently verified.

5. Model comparison

The table reports candidate algorithms. Selected marks the saved model; Tuned indicates an accepted tuning search. Values are rounded for readability.

Problem	Algorithm	Sel.	Tune	Acc.	F1	ROC-AUC	MAE	RMSE	R^2
Delivery delay	Random Forest tuned	Yes	Yes	0.829	0.905	0.512	—	—	—
Delivery delay	Decision Tree	No	No	0.542	0.681	0.487	—	—	—
Delivery delay	Logistic Regression	No	No	0.523	0.663	0.475	—	—	—
Product returns	Logistic Regression	Yes	No	0.538	0.163	0.488	—	—	—
Product returns	Decision Tree	No	No	0.421	0.147	0.476	—	—	—
Product returns	Random Forest	No	No	0.880	0.048	0.509	—	—	—
Product demand	Random Forest Regr. tuned	Yes	Yes	—	—	—	6.524	8.078	-0.014
Product demand	Decision Tree Regr.	No	No	—	—	—	6.658	8.485	-0.119
Product demand	Linear Regression	No	No	—	—	—	6.613	8.205	-0.046
Customer value	Linear Regression	Yes	No	—	—	—	2990.706	3757.004	-0.069
Customer value	Decision Tree Regr.	No	No	—	—	—	3133.165	4164.918	-0.313
Customer value	Random Forest Regr.	No	No	—	—	—	3034.185	3837.693	-0.115

The high accuracy of the product-return Random Forest candidate does not correspond to the selected model; its F1 is only 0.048. Accuracy must be interpreted alongside positive-class metrics for imbalanced classification.

6. Baseline comparisons

Classification baselines predict the majority class in the training partition. Regression baselines predict the mean target value from the training period.

Problem	Baseline	Acc.	Prec.	Recall	F1	MAE	RMSE	R^2
Delivery delay	Majority class	0.878	0.878	1.000	0.935	—	—	—
Product returns	Majority class	0.905	0.000	0.000	0.000	—	—	—
Product demand	Training-period mean	—	—	—	—	6.545	8.054	-0.008
Customer value	Training-period mean	—	—	—	—	2987.088	3747.890	-0.063

Delivery delay: the majority-class baseline achieved higher accuracy and F1 than the selected Random Forest in the supplementary run. Its recall of 1.000 comes from predicting the majority delayed class for every test record, not from strong discrimination. **Product returns:** the majority-class baseline achieved 90.5% accuracy but zero positive-class precision, recall, and F1; its accuracy does not demonstrate useful return detection. **Demand forecasting:** the selected model had slightly lower MAE than the baseline, but higher RMSE and lower R-squared, so improvement is mixed. **Customer value:** the mean baseline performed better on MAE, RMSE, and R-squared.

These are supplementary comparisons. The baseline script constructed its own split, and test rows have not been verified as identical to those used for the saved models. They should not be described as strict like-for-like comparisons until split alignment is confirmed.

7. Interpretation

The delivery-delay target is imbalanced, and its ROC-AUC is close to 0.5. Its high F1 should not be interpreted as strong ability to distinguish delayed from on-time shipments. The return classifier also has weak discrimination. Demand and next-month customer-value models have negative held-out R-squared, and the baselines show that selected models do not consistently improve on simple alternatives.

8. Feature importance and artefacts

Permutation importance is saved in `Models/Permutation_Importance.csv` and visualized in `Models/Feature_Importance.png`. Classification confusion matrices are saved under Models. The selected pipelines and feature definitions are stored in `Models/Best_Model.pkl`. The Streamlit app loads this bundle for interactive demonstrations. Supplementary EDA figures and CSV summaries are saved under Reports.

9. Assumptions and limitations

The delay target is defined from DelayDays because the supplied status field may disagree with the numeric delay field. A recorded return is treated as the return label. Supplier reliability is not modelled because supplier IDs are not directly linked to shipments. Warehouse capacity is absent. Inventory records contain repeated warehouse-product pairs without dated snapshots. Customer value is defined as next-month item revenue rather than lifetime value. The data covers a limited historical period and does not establish production-grade predictive performance. IQR outlier flags are statistical indicators and do not prove a value is erroneous.

10. Recommendations

Improve capture of planned and actual delivery times; reconcile delivery status with delay days; capture supplier IDs on purchase and shipment records; maintain time-stamped inventory snapshots; define return labels consistently; collect more historical periods; compare models using consistent documented splits; evaluate against simple baselines; investigate false positives and false negatives; and pilot predictions with human review before operational use.

11. Reproduction

Install dependencies from `App/requirements.txt` or follow the README. From the project root, run `python run_swiftlog_project.py` to regenerate processed datasets, model comparisons, model bundles, evaluation figures, reports, and presentation. This may overwrite generated outputs. Run `python enhance_swiftlog_eda.py` to regenerate supplementary EDA and baseline outputs, then rerun the notebook.